

Introduction to Electronic Design Automation

Homework 1

Student: 吳亞倫
ID: B13901011
Department: 電機工程學系

1 Design Methodology and Design Flow

1. (a)

Modeling

- **Brain signal model:** Model normal brain signals and seizure-related signals. This helps the system identify abnormal activity.
- **Seizure progression model:** Model how a seizure starts and develops over time. This helps the device detect seizure early and respond at the correct time.
- **Electrode-tissue interaction model:** Model how the electrodes sense neural signals and how electrical stimulation affects brain tissue.
- **Power and thermal model:** Model power consumption and heat generation, because the device is implanted in the brain and must remain safe.

Design

- **Sensing design:** Design electrodes and low-noise circuits to acquire weak brain signals clearly and reliably.
- **Detection design:** Design an algorithm or decision logic to determine whether seizure activity is happening or about to happen.
- **Control design:** Design when stimulation should be triggered and determine the proper pulse parameters, such as amplitude, duration, and frequency.
- **Implant hardware design:** Design the stimulator, battery, chip size, and package so the device can work safely for a long time inside the body.

Analysis

- **Detection performance analysis:** Analyze detection accuracy, false alarm rate, and detection latency.
- **Stimulation effectiveness and safety analysis:** Analyze whether stimulation can suppress seizure without damaging brain tissue.
- **Power analysis:** Analyze energy consumption and battery lifetime.
- **System-level analysis:** Analyze whether sensing, detection, control, and stimulation can operate correctly together in real time.

1. (b)

For each components of the CNN system, I suggest components should be implemented as follows:

1. Image pre-processing (Software / CPU):

This stage includes image capture handling, resizing, normalization, and format conversion. It is better placed in software because it requires flexibility and is easier to modify when the input format or application setting changes. Its computation cost is also lower than the main CNN layers.

2. Convolutional layers (Hardware / FPGA or ASIC):

Convolutional layers should be implemented in hardware because they dominate the computation of the CNN. They require a large number of multiply-accumulate operations and have strong parallelism. Hardware acceleration can greatly improve throughput, reduce latency, and save power.

3. Activation functions, ReLU (Hardware / FPGA or ASIC):

ReLU is simple and highly regular. It is applied many times after convolutional layers, so placing it in hardware is efficient.

4. Pooling layers (Hardware / FPGA or ASIC):

Pooling layers should also be implemented in hardware because they are part of the repeated feature extraction pipeline. Their operations are simple and regular, which makes them suitable for efficient streaming hardware design. This helps maintain high throughput.

5. Fully connected layers (Hardware / FPGA or ASIC):

Fully connected layers involve many dense arithmetic operations, especially when the layer size is large. Therefore, they are also suitable for hardware acceleration. Implementing them in hardware improves inference speed and energy efficiency under real-time constraints.

6. Output stage, softmax + classification (Software / CPU):

The output stage has much lower computation cost than convolutional and fully connected layers. It also includes final decision logic, which may need to be adjusted depending on the application. Therefore, it is more suitable to place this part in software for better flexibility and easier debugging.

7. Overall control and scheduling (Software / CPU):

The CPU should manage system control, memory/buffer handling, task scheduling, and communication between modules. These tasks involve control logic rather than massive parallel computation, so software is the more suitable choice.

1. (c)

Main synthesis tasks in VLSI design flow include system specification, functional design, logic synthesis, circuit design, physical design and verification, fabrication, and packaging. In each of the tasks, the design is represented as follows:

1. **System specification:** The design is represented as specification. At this stage, it describes system requirements, functions, performance goals, and interfaces at a very high level.
2. **Functional design:** The design is represented as behavioral representation. At this stage, the system is described by functional blocks, algorithms, FSM, datapath, or behavioral HDL.

3. **Logic synthesis:** The design is converted into structural representation at the logic level. The result is a gate-level netlist, composed of logic gates and flip-flops.
4. **Circuit design:** The design is still a structural representation, but now at the transistor level. The result is a transistor-level circuit / schematic built with MOS transistors and their connections.
5. **Physical design and verification:** The design is converted into physical representation. At this stage, it becomes chip layout data, including floorplan, placement, routing, and verification such as DRC/LVS.
6. **Fabrication:** The physical layout is transferred into silicon manufacturing. The design is represented as masks / wafer patterns used to fabricate the chip.
7. **Packaging:** The fabricated die is packaged into a usable IC. The design is represented as the final packaged chip ready for testing and system use.

2 Algorithm and Complexity

2. (a)

Let $g_n(x) = x^n$. Then $g_n(x)$ satisfies the recurrence relation

$$g_n(x) = x \cdot g_{n-1}(x),$$

with base case

$$g_0(x) = 1.$$

By definition,

$$P_{n(x)} = P_{n-1}(x) + a_0 g_n(x).$$

For each recursive step, one multiplication is required to compute $g_n(x)$, and one multiplication together with one addition is required to compute $P_{n(x)}$. Hence, computing $P_{n(x)}$ takes $2n$ multiplications and n additions overall, so the total time complexity is $O(n)$.

2. (b)

First, for each recursive step of calculating $P'_i(x) = xP'_{i-1}(x) + a_i$, one multiplication and one addition are required. Hence, calculating $P'_{n-1}(x)$ takes $n-1$ multiplications and $n-1$ additions overall, so the time complexity is $O(n-1)$.

Next, to calculate $P_n(x) = xP'_n(x) + a_0$, we need one multiplication and one addition. Hence, calculating $P_n(x)$ takes n multiplications and n additions overall, so the time complexity is $O(n)$.

3 NP-Completeness and Polynomial-Time Reduction

3. (a)

To prove that the feasibility problem of 0,1-ILP is NP-complete, it suffices to show that it is in NP and that it is NP-hard.

First, the problem is in NP. Given an assignment to the binary variables, we can verify in polynomial time whether all linear constraints are satisfied by simply substituting the

assigned values into each constraint and checking its validity. Since the number of constraints is polynomial in the input size, the verification procedure runs in polynomial time.

Next, we show that the problem is NP-hard by giving a polynomial-time reduction from 3-SAT. Consider an arbitrary 3-SAT instance with variables $x_1, x_2, \dots, x_n$ and clauses $C_1, C_2, \dots, C_m$. We construct a corresponding 0,1-ILP instance as follows.

- For each Boolean variable x_i , introduce a binary variable $y_i \in \{0, 1\}$.
- For each clause $C_j = (a \vee b \vee c)$, where a, b , and c are literals (can be x_i or $\neg x_i$), introduce the linear constraint

$$y_a + y_b + y_c \geq 1,$$

where $y_a = y_i$ if $a = x_i$, and $y_a = 1 - y_i$ if $a = \neg x_i$. The definitions of y_b and y_c are analogous.

This construction ensures that each clause is satisfied if and only if the corresponding linear constraint is satisfied. Therefore, the original 3-SAT formula is satisfiable if and only if the constructed 0,1-ILP instance is feasible. Moreover, the transformation clearly requires only polynomial time.

Hence, 0,1-ILP feasibility is NP-hard. Since it is both in NP and NP-hard, it follows that the feasibility problem of 0,1-ILP is NP-complete.

3. (b)

To prove that Circuit-SAT is NP-complete, we show that it is in NP and NP-hard.

First, Circuit-SAT is in NP. Given an assignment to the input variables, we can evaluate the circuit gate by gate in polynomial time and check whether the output is 1.

Next, we prove NP-hardness by reducing 0,1-ILP to Circuit-SAT. Consider an instance of 0,1-ILP: $\mathbf{Ax} \leq \mathbf{b}$, where each $x_i \in \{0, 1\}$. That is, the instance consists of inequalities

$$a_{j1}x_1 + a_{j2}x_2 + \dots + a_{jn}x_n \leq b_j,$$

for $j = 1, 2, \dots, m$.

For each inequality, we build a subcircuit. Since each x_i is binary, each term $a_{ji}x_i$ is either 0 or a_{ji} , so it can be produced by a small selector subcircuit. We then use binary adder circuits to sum all these terms and obtain the left-hand side of the inequality. After that, we use a comparator circuit to check whether this sum is at most b_j . Thus, the subcircuit outputs 1 if and only if the inequality is satisfied.

We repeat this construction for every inequality and connect all resulting outputs to a final AND gate. Therefore, the whole circuit outputs 1 if and only if all inequalities are satisfied, that is, if and only if the original 0,1-ILP instance is feasible.

This transformation can be carried out in polynomial time. Indeed, each selector, adder, and comparator circuit has size polynomial in the bit-length of the coefficients and constants, and the number of inequalities and variables is polynomial in the input size. Therefore, the entire circuit has polynomial size and can be constructed in polynomial time.

Hence, we have a polynomial-time reduction from 0,1-ILP to Circuit-SAT. Therefore, Circuit-SAT is NP-hard. Since it is also in NP, it follows that Circuit-SAT is NP-complete.

4 SAT and Integer Linear Programming

In this k -coloring problem, the graph $G(V, E)$ is shown as Figure 1 below, where $V = \{1, 2, 3, 4, 5, 6, 7\}$ and $E = \{(1, 2), (2, 3), (3, 4), (4, 5), (5, 1), (1, 6), (3, 6), (2, 7), (4, 7)\}$.

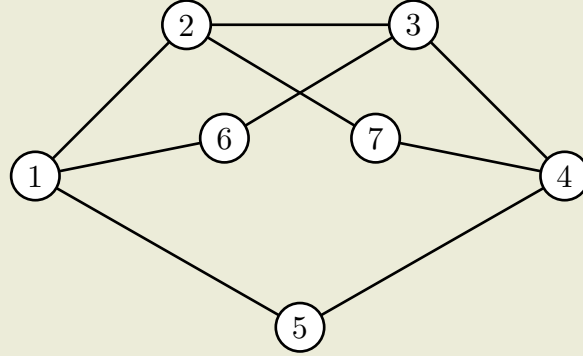


Figure 1: Graph for the k -coloring problem

4. (a)

To reduce the k -coloring problem to SAT, we introduce Boolean variables and construct a Boolean formula that is satisfiable if and only if the graph can be colored with k colors.

Case $k = 2$

First, considering $k = 2$ case. Let c_i be a Boolean variable indicating the color assigned to vertex i , where $c_i \in \{0, 1\}$.

For every edge $(i, j) \in E$, the two endpoints must have different colors, so we require $c_i \neq c_j$. This is equivalent to the CNF formula

$$(c_i \vee c_j) \wedge (\neg c_i \vee \neg c_j).$$

Therefore, the Boolean formula for the 2-coloring problem is:

$$f_2 = \bigwedge_{(i,j) \in E} ((c_i \vee c_j) \wedge (\neg c_i \vee \neg c_j)).$$

Referring to one [reference article](#) about MiniSat, we need to express the formula in a specific format file, so called DIMACS CNF. The first is the header line, which specifies the number of variables and clauses in the formula. From the 2nd line onwards, we list all the clauses in the formula. The following is my input file for the 2-coloring problem and the result is show in Figure 2 below.

2-coloring problem MiniSat input file

```
p cnf 7 18
1 2 0
-1 -2 0
2 3 0
-2 -3 0
3 4 0
-3 -4 0
4 5 0
-4 -5 0
5 1 0
-5 -1 0
```

```

1 6 0
-1 -6 0
3 6 0
-3 -6 0
2 7 0
-2 -7 0
4 7 0
-4 -7 0

```

```

> minisat 2colorsat.txt
===== [ Problem Statistics ] =====
|
| Number of variables:      7
| Number of clauses:      18
| Parse time:              0.00 s
| Eliminated clauses:      0.00 Mb
| Simplification time:     0.00 s
|
|=====
Solved by simplification
restarts      : 0
conflicts     : 0          (0 /sec)
decisions    : 0          ( nan % random) (0 /sec)
propagations  : 1          (434 /sec)
conflict literals : 0      ( nan % deleted)
Memory used   : 4.52 MB
CPU time      : 0.002305 s

UNSATISFIABLE

```

Figure 2: SAT solver result for the 2-coloring problem

Case $k = 3$

For the $k = 3$ case, we use a different encoding. Let $c_{i,c}$ be a Boolean variable indicating whether vertex i is colored with color c , where $c \in \{0, 1, 2\}$.

We need to ensure that each vertex is assigned at least one color, that is, for each vertex i , we require

$$(c_{i,0} \vee c_{i,1} \vee c_{i,2}).$$

We also need to ensure that each vertex is assigned at most one color, that is, for each vertex i and for each pair of distinct colors c and c' , we require

$$(\neg c_{i,c} \vee \neg c_{i,c'}).$$

Finally, we need to ensure that adjacent vertices have different colors. For each edge $(i, j) \in E$ and for each color c , we require

$$(\neg c_{i,c} \vee \neg c_{j,c}).$$

Hence, the Boolean formula for the 3-coloring problem is:

$$\begin{aligned}
f_3 = & \left(\bigwedge_{i \in V} (x_{i,0} \vee x_{i,1} \vee x_{i,2}) \right) \\
& \wedge \left(\bigwedge_{i \in V} ((\neg x_{i,0} \vee \neg x_{i,1}) \wedge (\neg x_{i,0} \vee \neg x_{i,2}) \wedge (\neg x_{i,1} \vee \neg x_{i,2})) \right) \\
& \wedge \left(\bigwedge_{(i,j) \in E} \bigwedge_{c=0}^2 (\neg x_{i,c} \vee \neg x_{j,c}) \right)
\end{aligned}$$

I also create the input file for the 3-coloring problem ($x_{i,c}$ is assign to $3i - c$ in the DIMACS CNF format) and the result is show in Figure 3 below.

3-coloring problem MiniSat input file

```

p cnf 21 55
1 2 3 0
4 5 6 0
7 8 9 0
10 11 12 0
13 14 15 0
16 17 18 0
19 20 21 0
-1 -2 0
-1 -3 0
-2 -3 0
-4 -5 0
-4 -6 0
-5 -6 0
-7 -8 0
-7 -9 0
-8 -9 0
-10 -11 0
-10 -12 0
-11 -12 0
-13 -14 0
-13 -15 0
-14 -15 0
-16 -17 0
-16 -18 0
-17 -18 0
-19 -20 0
-19 -21 0
-20 -21 0
-3 -6 0
-2 -5 0
-1 -4 0
-6 -9 0
-5 -8 0
-4 -7 0
-9 -12 0
-8 -11 0
-7 -10 0

```

```

-12 -15 0
-11 -14 0
-10 -13 0
-15 -3 0
-14 -2 0
-13 -1 0
-3 -18 0
-2 -17 0
-1 -16 0
-9 -18 0
-8 -17 0
-7 -16 0
-6 -21 0
-5 -20 0
-4 -19 0
-12 -21 0
-11 -20 0
-10 -19 0

```

```

> minisat 3colorsat.txt
===== [ Problem Statistics ] =====
|
| Number of variables:      21
| Number of clauses:       55
| Parse time:              0.00 s
| Eliminated clauses:      0.00 Mb
| Simplification time:     0.00 s
|
===== [ Search Statistics ] =====
| Conflicts |          ORIGINAL          |      LEARNT      | Progress |
|           | Vars  Clauses Literals |   Limit  Clauses Lit/Cl |           |
=====
restarts      : 1
conflicts     : 0              (0 /sec)
decisions     : 1              (0.00 % random) (383 /sec)
propagations  : 0              (0 /sec)
conflict literals : 0          ( nan % deleted)
Memory used   : 4.57 MB
CPU time      : 0.00261 s

SATISFIABLE

```

Figure 3: SAT solver result for the 3-coloring problem

4. (b)

To reduce the k -coloring problem to Integer Linear Programming (ILP), we introduce the following variables. For each vertex $i \in V$ and each color $c \in \{1, 2, \dots, k\}$, we introduce a binary variable $x_{i,c}$, where $x_{i,c} = 1$ if vertex i is colored with color c , and 0 otherwise.

We need to ensure that each vertex is assigned exactly one color, which can be expressed as the following constraints:

$$\sum_{c=1}^k x_{i,c} = 1, \forall i \in V.$$

We also need to ensure that adjacent vertices have different colors. For each edge $(i, j) \in E$ and for each color $c \in \{1, 2, \dots, k\}$, we can express this constraint as:

$$x_{i,c} + x_{j,c} \leq 1.$$

The above constraints can be combined to form the ILP formulation of the k -coloring problem. The objective function can be arbitrary (e.g., minimize 0) since we are only interested in feasibility.

By reading the documentation of `lp_solve`, we can write the ILP formulation in its input format. The following is my input file for the 2-coloring problem and the result is shown in Figure 4 below.

2-coloring problem lp_solve input file

```
min: 0;

x_1_1 + x_1_2 = 1;
x_2_1 + x_2_2 = 1;
x_3_1 + x_3_2 = 1;
x_4_1 + x_4_2 = 1;
x_5_1 + x_5_2 = 1;
x_6_1 + x_6_2 = 1;
x_7_1 + x_7_2 = 1;

x_1_1 + x_2_1 <= 1; x_1_2 + x_2_2 <= 1;
x_2_1 + x_3_1 <= 1; x_2_2 + x_3_2 <= 1;
x_3_1 + x_4_1 <= 1; x_3_2 + x_4_2 <= 1;
x_4_1 + x_5_1 <= 1; x_4_2 + x_5_2 <= 1;
x_5_1 + x_1_1 <= 1; x_5_2 + x_1_2 <= 1;
x_1_1 + x_6_1 <= 1; x_1_2 + x_6_2 <= 1;
x_3_1 + x_6_1 <= 1; x_3_2 + x_6_2 <= 1;
x_2_1 + x_7_1 <= 1; x_2_2 + x_7_2 <= 1;
x_4_1 + x_7_1 <= 1; x_4_2 + x_7_2 <= 1;

bin x_1_1, x_1_2, x_2_1, x_2_2, x_3_1, x_3_2, x_4_1,
    x_4_2, x_5_1, x_5_2, x_6_1, x_6_2, x_7_1, x_7_2;
```

```
> lp_solve 2color.lp
This problem is infeasible
```

Figure 4: ILP solver result for the 2-coloring problem

The following is my input file for the 3-coloring problem and the result is shown in Figure 5 below.

3-coloring problem lp_solve input file

```
min: 0;

x_1_1 + x_1_2 + x_1_3 = 1;
x_2_1 + x_2_2 + x_2_3 = 1;
x_3_1 + x_3_2 + x_3_3 = 1;
x_4_1 + x_4_2 + x_4_3 = 1;
x_5_1 + x_5_2 + x_5_3 = 1;
```

```

x_6_1 + x_6_2 + x_6_3 = 1;
x_7_1 + x_7_2 + x_7_3 = 1;

x_1_1 + x_2_1 <= 1; x_1_2 + x_2_2 <= 1; x_1_3 + x_2_3 <= 1;
x_2_1 + x_3_1 <= 1; x_2_2 + x_3_2 <= 1; x_2_3 + x_3_3 <= 1;
x_3_1 + x_4_1 <= 1; x_3_2 + x_4_2 <= 1; x_3_3 + x_4_3 <= 1;
x_4_1 + x_5_1 <= 1; x_4_2 + x_5_2 <= 1; x_4_3 + x_5_3 <= 1;
x_5_1 + x_1_1 <= 1; x_5_2 + x_1_2 <= 1; x_5_3 + x_1_3 <= 1;
x_1_1 + x_6_1 <= 1; x_1_2 + x_6_2 <= 1; x_1_3 + x_6_3 <= 1;
x_3_1 + x_6_1 <= 1; x_3_2 + x_6_2 <= 1; x_3_3 + x_6_3 <= 1;
x_2_1 + x_7_1 <= 1; x_2_2 + x_7_2 <= 1; x_2_3 + x_7_3 <= 1;
x_4_1 + x_7_1 <= 1; x_4_2 + x_7_2 <= 1; x_4_3 + x_7_3 <= 1;

bin x_1_1, x_1_2, x_1_3, x_2_1, x_2_2, x_2_3, x_3_1, x_3_2, x_3_3,
    x_4_1, x_4_2, x_4_3, x_5_1, x_5_2, x_5_3, x_6_1, x_6_2, x_6_3,
    x_7_1, x_7_2, x_7_3;

```

```

> lp_solve 3color.lp

Value of objective function: 0

Actual values of the variables:
x_1_1      0
x_1_2      0
x_1_3      1
x_2_1      0
x_2_2      1
x_2_3      0
x_3_1      0
x_3_2      0
x_3_3      1
x_4_1      0
x_4_2      1
x_4_3      0
x_5_1      1
x_5_2      0
x_5_3      0
x_6_1      1
x_6_2      0
x_6_3      0
x_7_1      1
x_7_2      0
x_7_3      0

```

Figure 5: ILP solver result for the 3-coloring problem

5 Model of Computation

5. (a)

For the regular expression

$$(01^*0)^*1,$$

we construct a DFA as follows. State q_0 is the start state. Reading 0 starts a loop and moves the automaton to q_1 , where it may read any number of 1's. Reading 0 from q_1 completes the loop and returns to q_0 . Reading 1 from q_0 corresponds to the final symbol of the string, so the automaton moves to the accepting state q_2 .

Since a DFA must define a transition for every input symbol at every state, we add a trap state q_t . Any input read after reaching q_2 leads to q_t , and q_t loops on both 0 and 1.

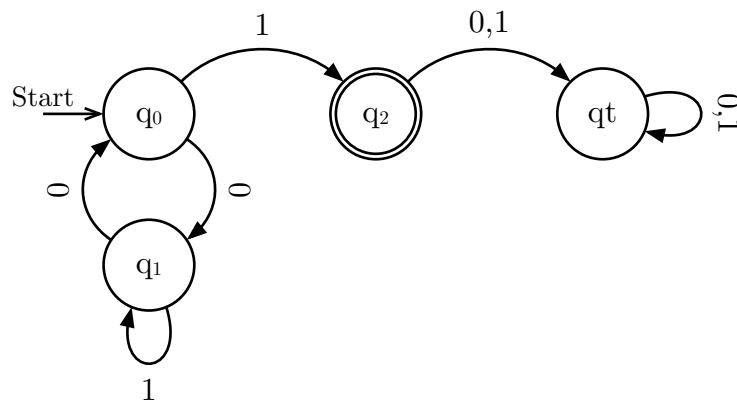


Figure 6: DFA for the regular expression $(01^*0)^*1$

5. (b)

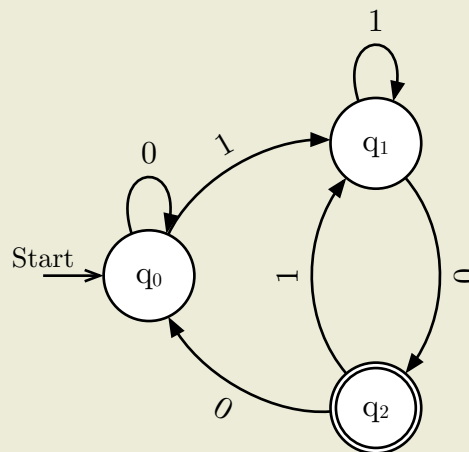


Figure 7: The finite automaton given in the problem statement

To derive the regular expression for the language accepted by the given automaton, we analyze the paths:

- First time from q_0 to q_2 :
 1. zero or many 0
 2. one or many 1
 3. one 0

► The regex for this part is 0^*1^+0 .

- Loop back from q_2 to q_0 to q_2 :
 1. one or many 0
 2. one or many 1
 3. one 0
 ▶ The regex for this part is 0^+1^+0 .
- Loop back from q_2 to q_1 to q_2 :
 1. one or many 1
 2. one 0
 ▶ The regex for this part is 1^+0 .

Combining the looping back parts, we have

$$(0^+1^+0) \cup (1^+0) = 0^*1^+0,$$

so the regex for the language accepted by the automaton is

$$0^*1^+0(0^*1^+0)^* = (0^*1^+0)^+$$

5. (c)

The required timed automaton is as follows. The label RE stands for “Requesting” and AG stands for “Access Granted”.

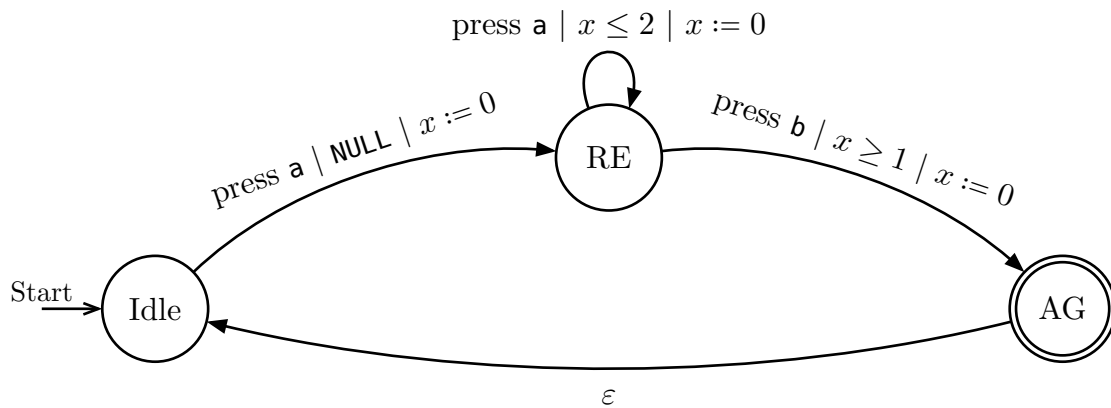


Figure 8: Timed automaton for the timed access control system

5. (d)

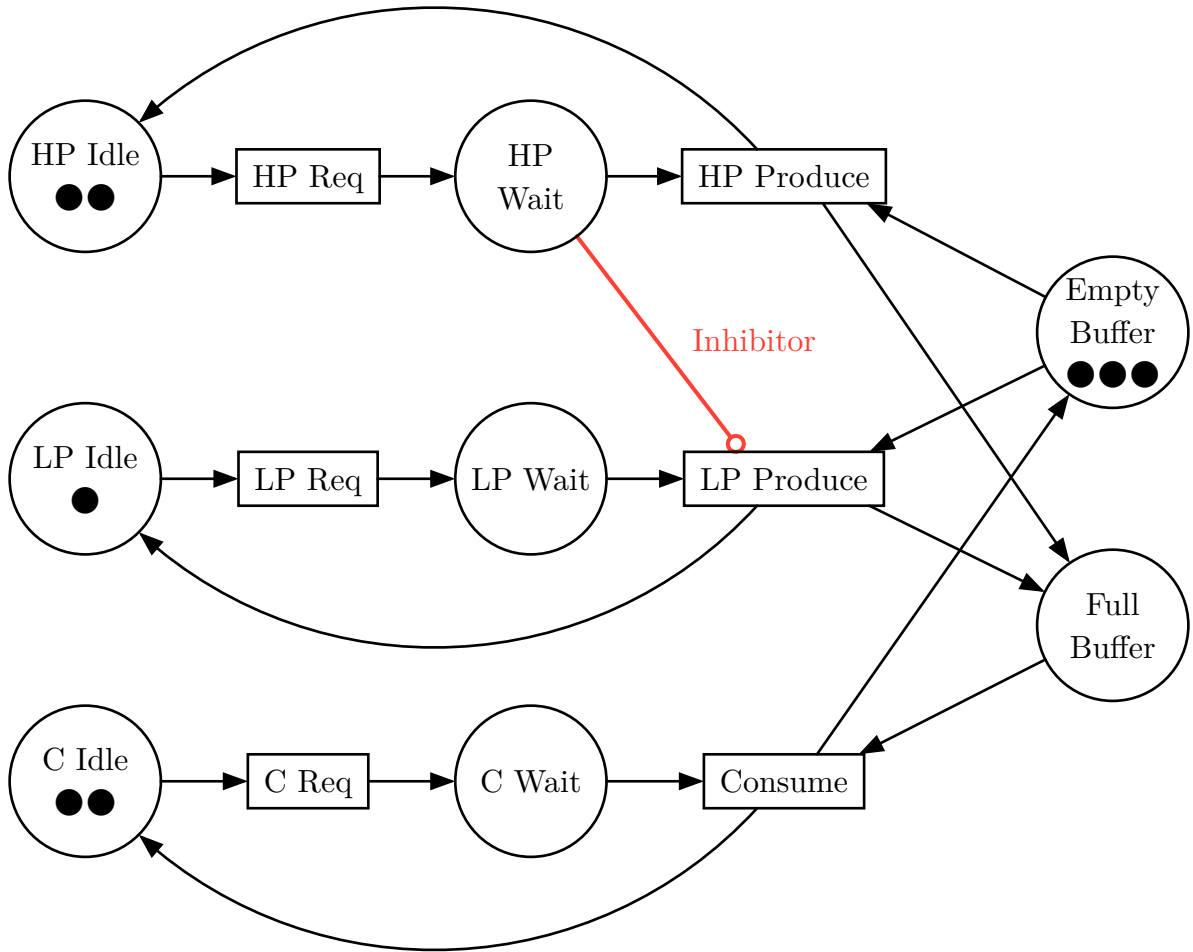


Figure 9: The Petri net of the producer-consumer system. Here, HP, LP, and C denote the high-priority producer, low-priority producer, and consumer, respectively, while Empty Buffer and Full Buffer represent the numbers of available and occupied buffer slots. The inhibitor arc from HP Wait to LP Produce enforces the priority policy by disabling LP Produce whenever a high-priority request is waiting. This design ensures that HP production is always served before LP production when both contend for the shared buffer.

6 Scheduling

6. (a) Data-Flow Graph (DFG)

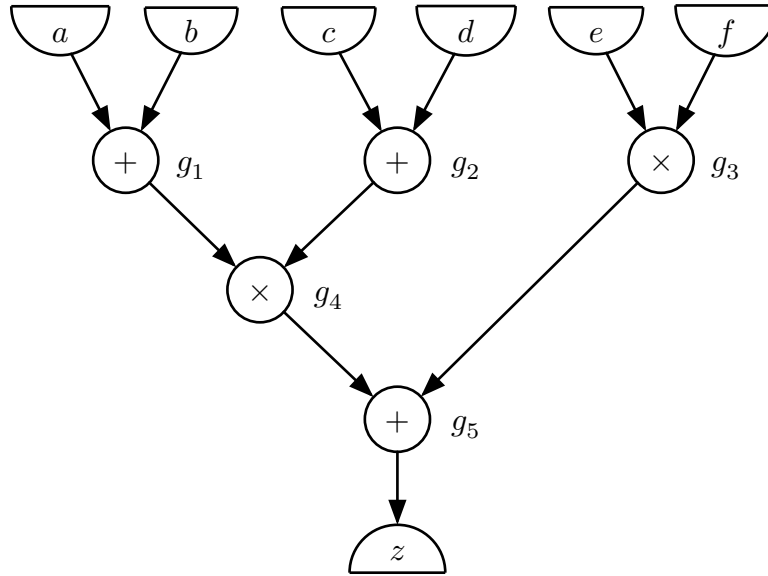


Figure 10: The corresponding DFG of $z = (a + b) \times (c + d) + (e \times f)$

6. (b) ASAP Scheduling

After ASAP scheduling, we need total 4 clock cycles to finish the computation. Additionally, we need 2 adders and 2 multipliers to achieve this schedule.

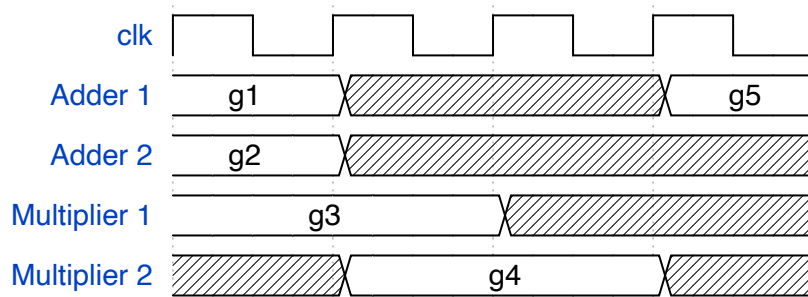


Figure 11: ASAP scheduling result time chart

6. (c) ALAP Scheduling

After ALAP scheduling, we need total 4 clock cycles to finish the computation. Additionally, we need 2 adders and 2 multipliers to achieve this schedule.

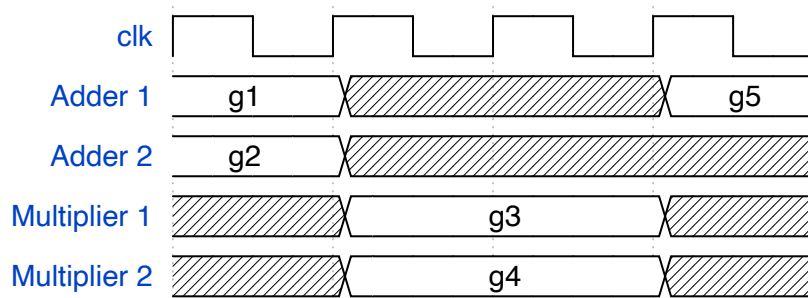


Figure 12: ALAP scheduling result time chart

6. (d) Mobility-based Scheduling

- g_1 : ASAP = 1, ALAP = 1, mobility = 0
- g_2 : ASAP = 1, ALAP = 1, mobility = 0
- g_3 : ASAP = 1, ALAP = 2, mobility = 1
- g_4 : ASAP = 2, ALAP = 2, mobility = 0
- g_5 : ASAP = 4, ALAP = 4, mobility = 0

For this mobility, we cannot further reduce the number of functional units, so we still need 2 adders and 2 multipliers. However, we can schedule g_3 at either cycle 1 or cycle 2, so there are two possible schedules.

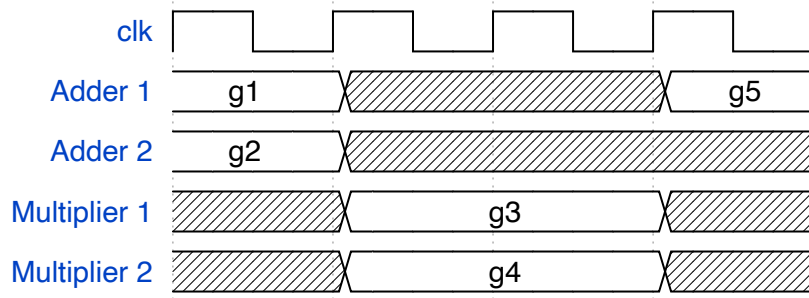


Figure 13: Mobility-based scheduling result time chart

6. (e) Critical-path list scheduling

- Priority: ($P(\cdot)$ denotes the priority of a node, and $d(\cdot)$ denotes the delay of a node)
 - $P(g_1) = d(g_1) + d(g_4) + d(g_5) = 4$
 - $P(g_2) = d(g_2) + d(g_4) + d(g_5) = 4$
 - $P(g_3) = d(g_3) + d(g_5) = 3$
 - $P(g_4) = d(g_4) + d(g_5) = 3$
 - $P(g_5) = d(g_5) = 1$
- Schedule:
 - Cycle 1: Ready: g_1, g_2, g_3 . Schedule g_1 and g_2 to A_1 and A_2 .
 - Cycle 2: Ready: g_3, g_4 . Schedule g_3 to A_1 .
 - Cycle 4: Ready: g_4 . Schedule g_4 to A_1 .
 - Cycle 5: Ready: g_5 . Schedule g_5 to A_1 .

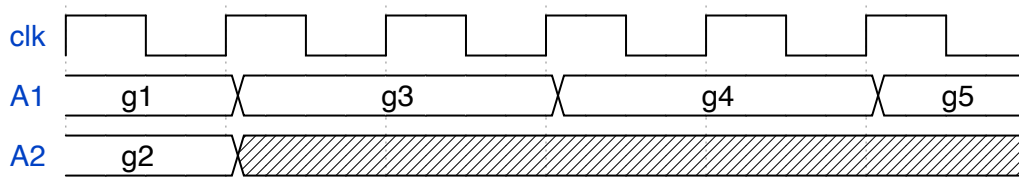


Figure 14: Critical-path list scheduling result time chart